

Documento de Visão do Produto: CodeParfum Atendente Automatizado

Florindo Rian Silva Carreteiro

florindo.carreteiro@edu.ufopa.br

Semestre: 2025.2

Disciplina: Sistemas Distribuídos

Docente: Rennan José Maia Da Silva

Repositório:

<https://github.com/florindorian/DISC-SISTD-final-evolutionapi>

1 Introdução

Este Documento de Visão descreve o **CodeParfum Atendente Automatizado**, uma solução de atendimento ao cliente via WhatsApp. O sistema é concebido como um **template de arquitetura distribuída** minimalista e modular, cujo foco é a **automação de processos de negócio** para consultas no catálogo da perfumaria CodeParfum.

O projeto demonstra a integração desacoplada de algumas camadas distintas, comunicando-se via Webhooks e APIs REST, um princípio fundamental da disciplina de Sistemas Distribuídos. A inteligência é fornecida pelo *Large Language Model* (LLM) **Gemini**, e o motor de dados é o **MongoDB** (NoSQL). A arquitetura reformulada do projeto, expressa aqui por seu diagrama de componentes, pode ser observada na Figura X.

Atendente de Perfumaria via WhatsApp (Florindo R. S. Carreteiro)

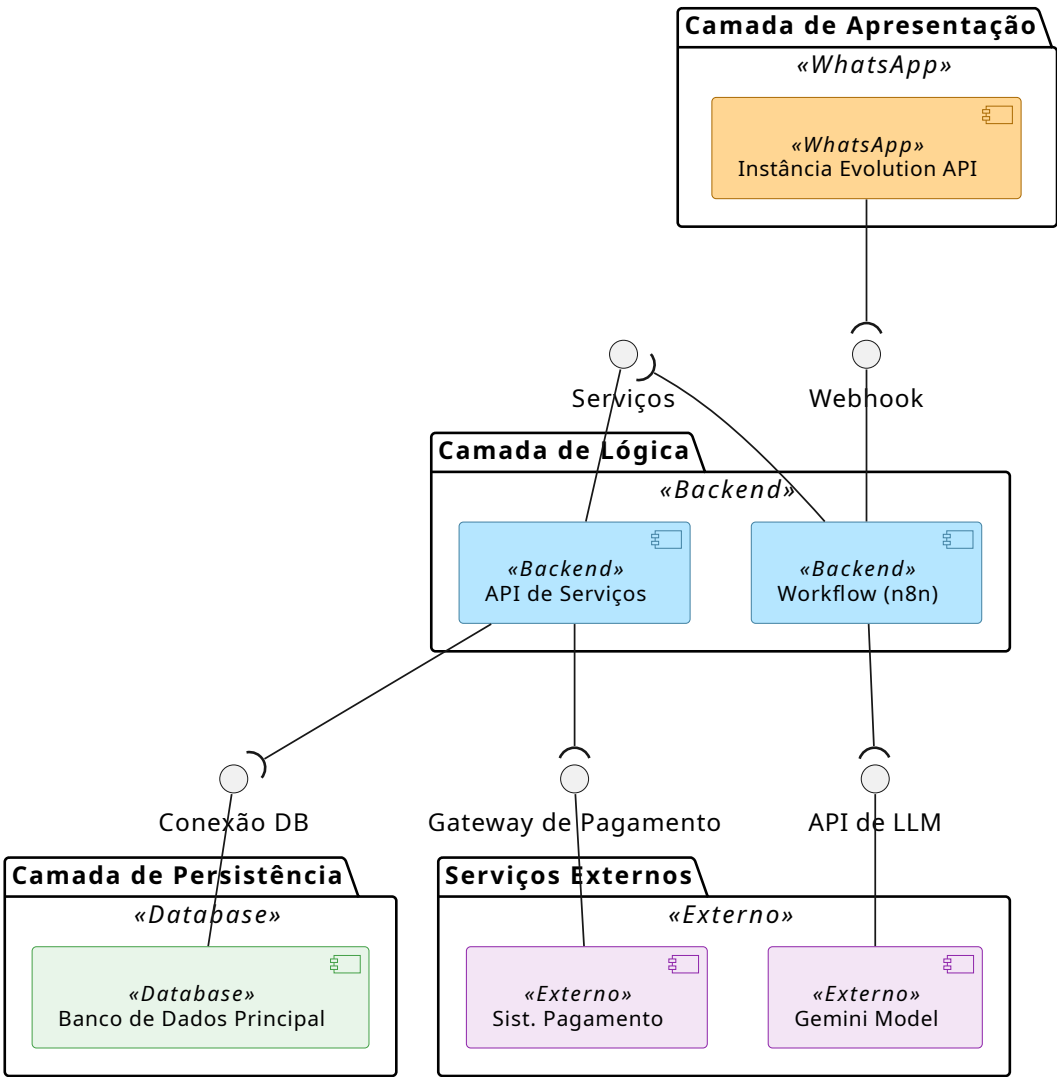


Figura 1: Diagrama de Componentes do CodeParfum Atendente Automatizado.

2 Posicionamento do Produto

2.1 Problema (As-Is)

Empresas de cosméticos que utilizam canais de mensagens para interagir com clientes geram um grande volume de dados textuais. Atualmente, a conversão dessas perguntas em respostas precisas,

baseadas em um catálogo complexo (que inclui *Ciclos*, *Categorias* e *Promoções*), é manual e ineficiente. Gargalos operacionais incluem:

- A dificuldade em extrair intenções e entidades do texto livre do cliente (ex: “*Quero saber o perfume da promoção do ciclo 3*”).
- A perda de contexto e a incapacidade de *acumular perguntas “picotadas”* para formar uma *query* de busca completa.

2.2 Oportunidade

Existe uma necessidade crítica de uma solução que possa, automaticamente e em tempo real, transformar o texto livre em uma ação acionável na **API de Serviços**, permitindo a consulta rápida e precisa ao catálogo **MongoDB** para execução de serviços como: *Consulta de catálogo*, *Conversar com vendedor sobre promoções*, *Pagar a compra* e *Atendimento*.

3 Stakeholders e Usuários

Stakeholder	Interesses Primários
Gerentes de Operações	Redução do tempo de resposta e aumento da eficiência no atendimento.
Usuários Finais (Clientes)	Atendimento mais rápido, mais preciso e resolução automatizada de suas necessidades.

Tabela 1: Stakeholders do Produto

4 Arquitetura Distribuída e Componentes Chave

O **CodeParfum Atendente Automatizado** adota uma arquitetura em camadas, com componentes independentes e desacoplados, conforme o diagrama de arquitetura (UML).

4.1 Princípios de Distribuição

- **Desacoplamento Temporal e Físico:** Cada componente (WhatsApp, n8n, API, MongoDB) pode ser escalado e atualizado de forma independente.
- **Comunicação Assíncrona/Síncrona:** A entrada de dados utiliza **Webhooks** (assíncrono) e a comunicação interna de serviços utiliza **API REST** (síncrona).

4.2 Camadas e Componentes

1. **Camada de Apresentação (WhatsApp / Evolution API):**
 - **Instância Evolution API (WhatsApp):** Ponto de entrada das interações do cliente.
 - **Interface:** Envia as mensagens do cliente para a Camada de Lógica via **Webhook**.
2. **Camada de Lógica (API de Serviços, n8n):**
 - **Workflow (n8n):** Atua como o motor de automação e **orquestração** central. Recebe o *Webhook*, processa a mensagem com o Gemini, mantém o estado da conversa e aciona a API de Serviços para consultas no MongoDB.
 - **API de Serviços:** Microserviço dedicado a expor as consultas de catálogo. Recebe comandos do n8n e executa as *queries* complexas no MongoDB.
3. **Camada de Persistência (MongoDB):**
 - **Banco de Dados Principal (MongoDB):** Armazena o catálogo da CodeParfum (*Ciclo*, *Produto*, *Categoria*). Sua natureza NoSQL permite o armazenamento dos subdocumentos e relacionamentos complexos, otimizando as buscas exigidas pelo atendimento.

4. Serviços Externos (Gemini, Gateway):

- **Gemini Model:** Atua como um serviço externo desacoplado. Utilizado para **Atendimento Humanizado** e extração de *Intenção* e *Entidades* (*Marca*, *Ciclo*, *Categoria*) do texto do cliente.
- **Sist. Pagamento:** Gateway para simular a funcionalidade de *Pagar a compra*, acessado via **API REST**.

5 Funcionalidades Chave (Key Features)

O Atendente Automatizado deve entregar as seguintes capacidades mínimas (MVP) de automação e processamento.

5.1 Automação e Processamento (Gemini + n8n + MongoDB)

1. Atendimento Humanizado e Pausa:

- Utilização do **Gemini** para fornecer respostas em linguagem natural e direcionar a conversa para Setor 1 ou Setor 2.
- **Pausar** o fluxo do n8n quando o atendimento for finalizado.

2. Gestão de Contexto e *Query* Dinâmica:

- Capacidade de **acumular perguntas “picotadas”** para formar um contexto completo antes de acionar a busca no MongoDB.
- **Reconhecimento de Intenção e Entidades** via Gemini para mapear o pedido do cliente às *queries* do catálogo (ex: *marca*, *linha*, *ciclo*).

3. Execução de Serviços do Catálogo (via API de Serviços): O n8n orquestra a execução dos processos de negócio, acionando a **API de Serviços** para:

- **Consulta de catálogo:** Buscar produtos por *categoria*, *marca* e *linha* (`db.Produto.find(...)`).
- **Consulta de Promoções:** Localizar produtos em *promoção* em determinado(s) *ciclo(s)* (`db.Ciclo.aggregate(...)`).
- **Atendimento:** Marcar reunião / agendar visita / registrar reclamação (simulação de inserção em um módulo de CRM ou log).
- **Pagamento:** Acionar o *Gateway de Pagamento* para finalizar a compra.

4. Regra de Negócio: O sistema deve **Não atender a grupos** de WhatsApp, focando apenas em interações 1:1.